

Asignatura: Desarrollo Fullstack III (DSY1106)

Profesor: Cesar Poblete

Evaluación: Parcial N°1 – Diseño de Arquitecturas

Informe Técnico: Arquitectura de Microservicios – Grupo Cordillera

Integrantes: - Germán Maraboli
- Benjamin Monasterio
- Camila Vera

Índice

Índice	1
1. Descripción del Contexto y Necesidades del Cliente	2
Requerimientos Funcionales (RF)	2
Módulo 1 — Gestión de Datos Organizacionales	2
Módulo 2 — Gestión de Indicadores (KPI)	2
Módulo 3 — Visualización de Reportes	3
Requerimientos Técnicos con impacto funcional	3
Requerimientos No Funcionales (RNF)	3
2. Selección y Justificación de Patrones de Arquitectura	4
3. Herramientas y Estrategias de Implementación	4
4. Estructura de Microservicios (Diagrama de Arquitectura)	5
5. Seguridad, Privacidad y Sostenibilidad	8
6. Evaluación General del Diseño vs. Requerimientos	9

1. Descripción del Contexto y Necesidades del Cliente

Grupo Cordillera, empresa líder en el sector retail con presencia multiciudad, enfrenta una crisis de gestión de información debido a su rápido crecimiento. Actualmente, la data crítica (ventas, inventario, finanzas, clientes) reside en sistemas aislados ("silos de datos"), lo que obliga a los equipos a realizar procesos manuales de extracción y consolidación en hojas de cálculo.

Necesidades detectadas:

- **Centralización:** Eliminar la dispersión de datos en múltiples plataformas.
- **Tiempo Real:** Proveer indicadores clave (KPI) actualizados para la alta gerencia.
- **Escalabilidad:** Una base tecnológica que soporte la expansión a nuevos mercados sin degradar el rendimiento.
- **Fiabilidad:** Reducir el error humano derivado de la transcripción manual de reportes.

Requerimientos Funcionales (RF)

Módulo 1 — Gestión de Datos Organizacionales

RF-01 El sistema debe consolidar información proveniente de los sistemas de punto de venta (POS), plataformas de e-commerce, gestión de inventarios, gestión financiera y atención al cliente en un repositorio centralizado.

RF-02 El sistema debe estandarizar y normalizar los datos recibidos desde fuentes externas para garantizar su consistencia y calidad.

RF-03 El sistema debe eliminar la necesidad de procesos manuales de extracción, consolidación y transcripción de datos entre sistemas.

Módulo 2 — Gestión de Indicadores (KPI)

RF-04 El sistema debe permitir definir indicadores clave de desempeño (KPI) alineados con los objetivos estratégicos de la organización.

RF-05 El sistema debe calcular y actualizar en tiempo real métricas de ventas, inventario, logística y rentabilidad.

RF-06 El sistema debe comparar los indicadores calculados contra períodos o metas anteriores, permitiendo detectar variaciones porcentuales.

Módulo 3 — Visualización de Reportes

RF-07 El sistema debe proporcionar un panel de control (dashboard) que muestre los indicadores estratégicos de manera visual e intuitiva para la alta gerencia.

RF-08 El sistema debe generar reportes descargables con los datos consolidados según los requerimientos de los ejecutivos.

RF-09 El sistema debe permitir que solo usuarios con roles autorizados accedan a la información sensible (ventas, rentabilidad).

Requerimientos Técnicos con impacto funcional

RF-10 El sistema debe exponer una API Gateway como único punto de entrada, gestionando el enrutamiento y la seguridad mediante tokens JWT.

RF-11 Cada microservicio debe operar con su propia base de datos independiente (Database per Service) para garantizar autonomía de la data.

RF-12 La arquitectura debe permitir escalar servicios de forma selectiva (por ejemplo, solo MS-DATOS) sin afectar el resto del sistema.

Requerimientos No Funcionales (RNF)

RNF-01 — Rendimiento: El dashboard debe cargar los indicadores KPI en un tiempo máximo de 3 segundos bajo condiciones normales de operación.

RNF-02 — Escalabilidad: La arquitectura debe permitir escalar microservicios de forma independiente sin afectar la disponibilidad del resto del sistema.

RNF-03 — Seguridad: Todas las comunicaciones entre el cliente y el API Gateway deben estar autenticadas mediante tokens JWT.

RNF-04 — Disponibilidad: El sistema debe mantener operatividad ante fallos parciales mediante el patrón Circuit Breaker, evitando caídas en cascada.

2. Selección y Justificación de Patrones de Arquitectura

Para garantizar la escalabilidad y resiliencia del sistema, se han seleccionado los siguientes patrones:

- **API Gateway (Spring Cloud Gateway):** Punto de entrada único que centraliza la seguridad (JWT) y el enrutamiento. Es vital para desacoplar el frontend de los microservicios internos.
- **Backend For Frontend (BFF):** Capa intermedia que orquesta las llamadas a los microservicios y entrega al dashboard en React la data ya procesada y filtrada, optimizando el rendimiento.
- **Circuit Breaker:** Implementado en el Gateway para evitar caídas en cascada. Si un servicio de origen (como el POS) presenta latencia, el sistema sigue operativo.
- **Repository Pattern:** Aplicado en cada microservicio para separar la lógica de negocio de la persistencia en MySQL, asegurando un código limpio y mantenible.
- **Factory Method:** Utilizado en el módulo de reportes para generar dinámicamente distintos formatos de visualización según el requerimiento del usuario.

3. Herramientas y Estrategias de Implementación

Frontend: React.js, utilizando una arquitectura basada en componentes reutilizables para el Dashboard ejecutivo.

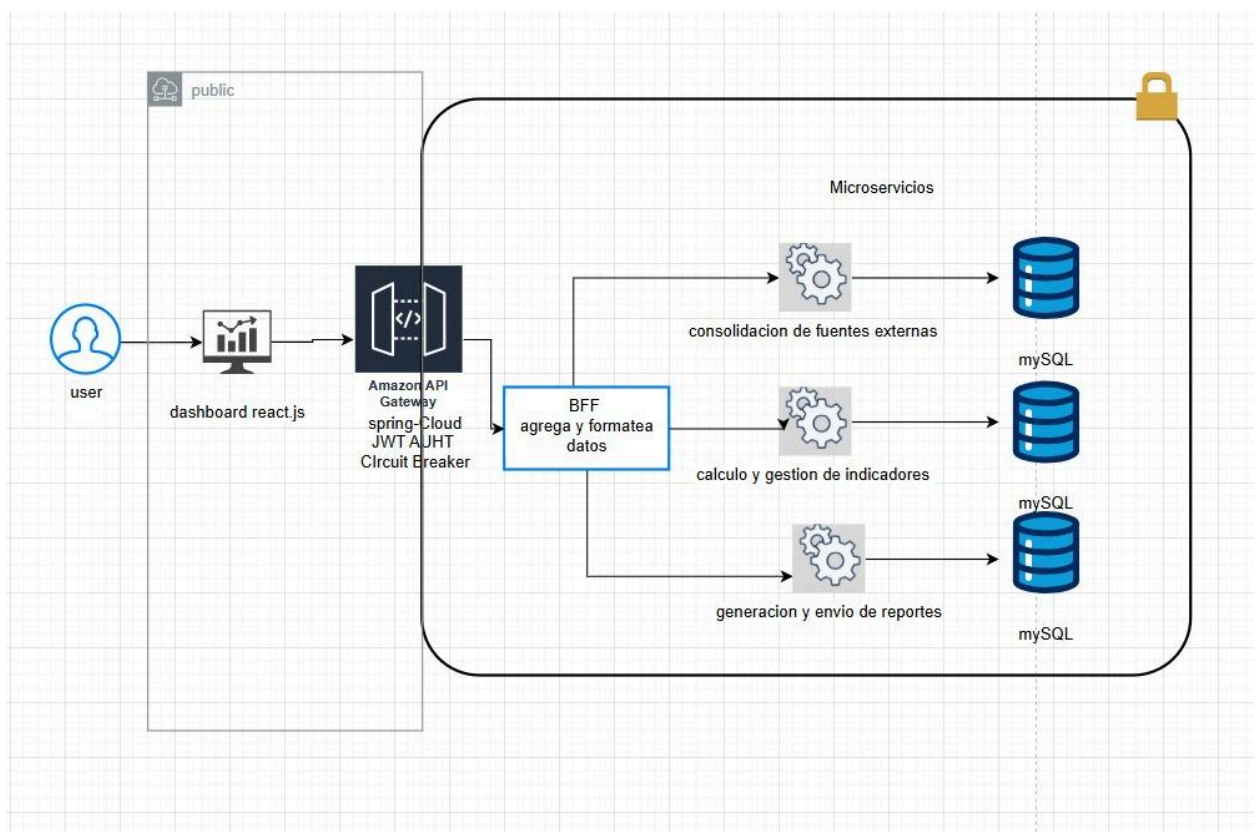
Backend: Spring Boot con Spring Cloud Gateway para la gestión de tráfico y seguridad.

Base de Datos: MySQL como motor relacional independiente para cada servicio (Database per Service), garantizando la autonomía de la data.

Integración de Datos: Se mantienen los Sistemas de Punto de Venta (POS) y E-commerce como fuentes externas que alimentan al sistema mediante procesos de ingesta controlados hacia el MS-DATOS.

Estrategia de Desarrollo: Uso de Maven para la gestión de dependencias y GitHub para el control de versiones colaborativo.

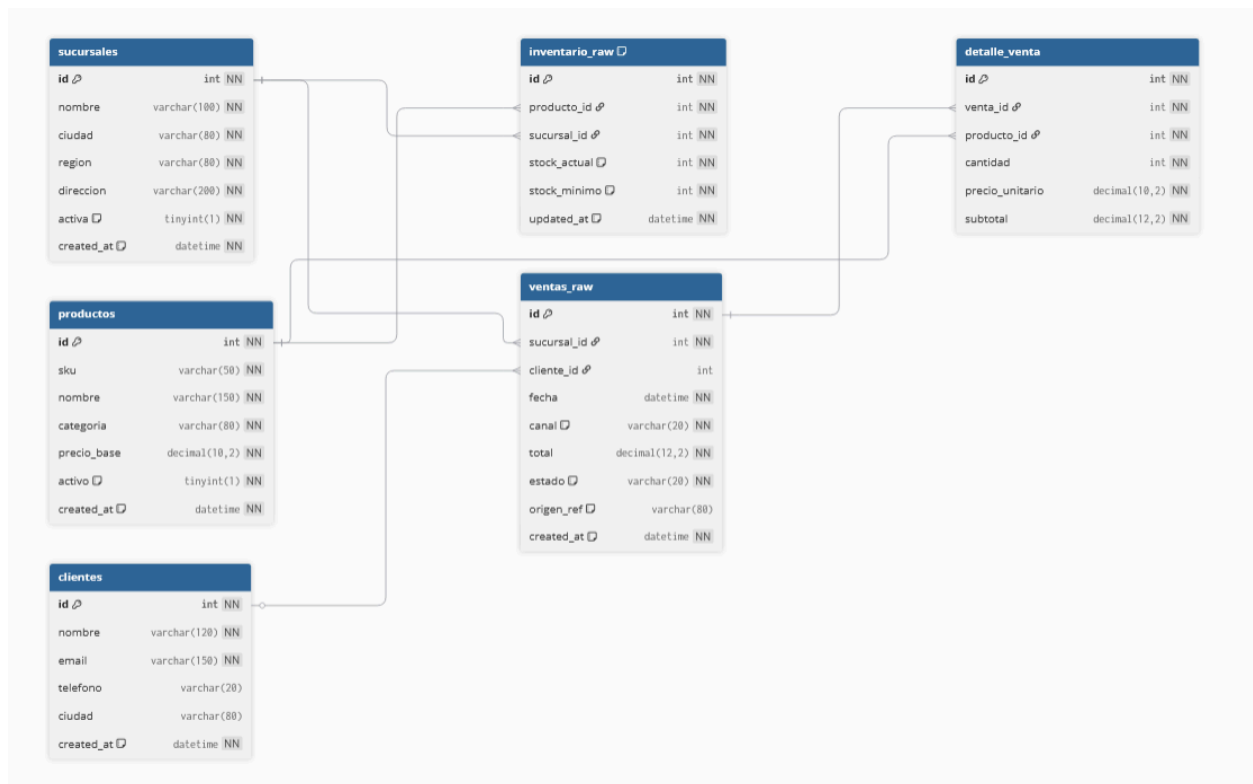
4. Estructura de Microservicios (Diagrama de Arquitectura)



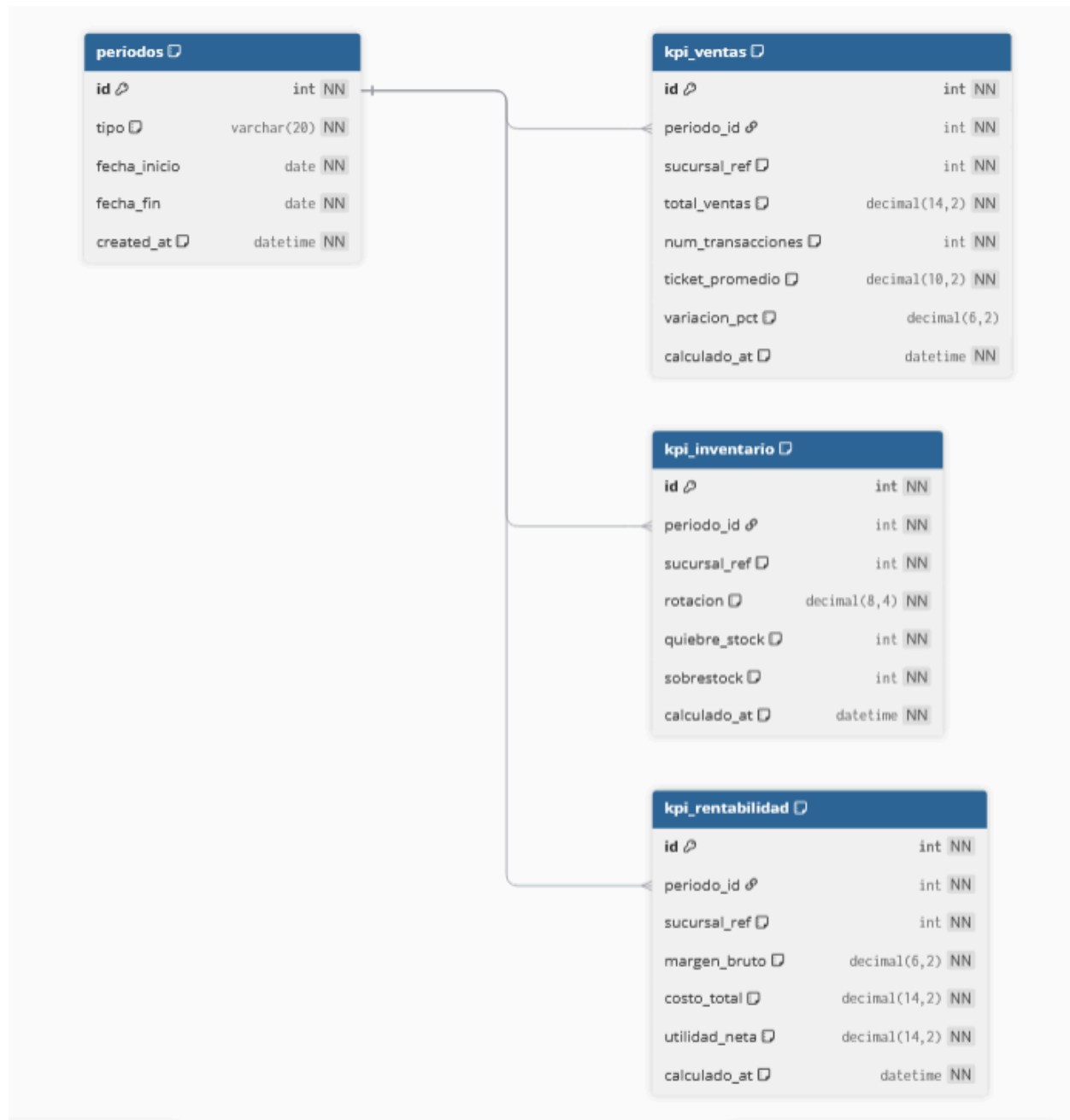
Detalle de los componentes en el diagrama:

1. **MS-DATOS:** Encargado de recibir y estandarizar la información proveniente de los POS y plataformas de E-commerce.
2. **MS-KPI:** Procesa la data del MS-DATOS para calcular métricas de ventas y rentabilidad.
3. **MS-REPORTES:** Genera las vistas finales y documentos descargables para la alta gerencia.

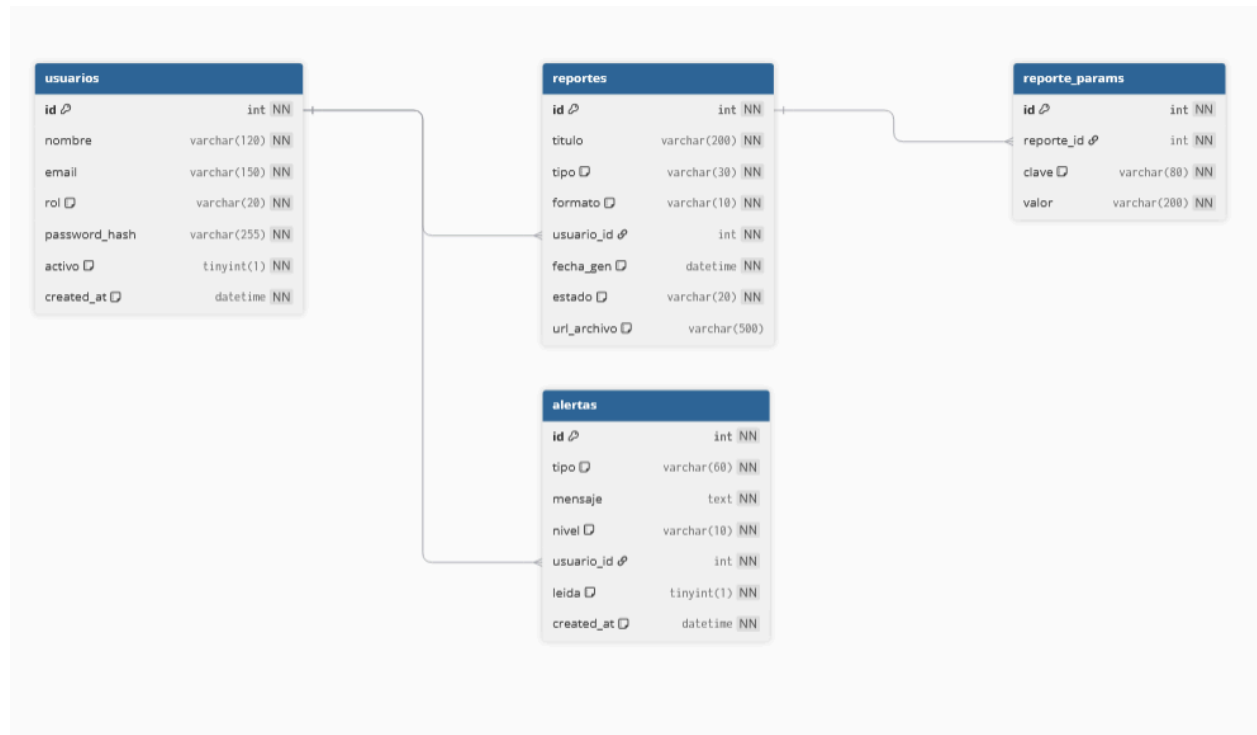
MS-DATOS



MS-KPI



MS-REPORTES



5. Seguridad, Privacidad y Sostenibilidad

Seguridad y Privacidad: El acceso a la información sensible (rentabilidad y ventas) está protegido en el API Gateway mediante tokens JWT, asegurando que solo los roles autorizados consuman los microservicios.

Sostenibilidad: La arquitectura permite el escalado selectivo. Si el flujo de datos desde los puntos de venta aumenta, solo se escala el MS-DATOS, optimizando el uso de servidores y reduciendo el consumo energético de la infraestructura.

Mantenibilidad: El uso de patrones de diseño y una estructura desacoplada asegura que el sistema pueda mantenerse y evolucionar a largo plazo sin necesidad de reconstrucciones completas.

6. Evaluación General del Diseño vs. Requerimientos

La arquitectura propuesta responde de manera satisfactoria a los requerimientos funcionales y técnicos definidos para Grupo Cordillera. A continuación se detalla cómo cada componente del diseño da cumplimiento a los requerimientos establecidos.

El MS-DATOS aborda directamente RF-01, RF-02 y RF-03, al centralizar la ingesta de información proveniente de los sistemas POS y e-commerce, estandarizando los datos antes de disponibilizarlos al resto de la arquitectura. Esto elimina los procesos manuales de consolidación que generaban errores y retrasos en los reportes ejecutivos.

El MS-KPI cumple con RF-04, RF-05 y RF-06, al calcular en tiempo real los indicadores clave de ventas, inventario y rentabilidad, organizados por período y sucursal. Las tablas `kpi_ventas`, `kpi_inventario` y `kpi_rentabilidad` evidencian la capacidad del sistema para comparar métricas entre períodos, cubriendo la necesidad de variación porcentual requerida por la alta gerencia.

El MS-REPORTES satisface RF-07 y RF-08, proporcionando vistas consolidadas y documentos descargables. La tabla `alertas` agrega además una capa proactiva de notificación no contemplada originalmente en los requerimientos, lo que representa un valor adicional para los usuarios ejecutivos.

Respecto a RF-09, RF-10 y RF-11, el API Gateway con autenticación JWT asegura que solo usuarios con roles autorizados accedan a la información sensible, mientras que el patrón `Database per Service` garantiza la autonomía e independencia de cada microservicio. Finalmente, RF-12 se satisface mediante la arquitectura desacoplada, que permite escalar selectivamente cualquier microservicio sin afectar la disponibilidad del sistema completo.

En cuanto a los requerimientos no funcionales, RNF-01 se aborda mediante el patrón `BFF`, que reduce la cantidad de llamadas al backend entregando datos ya procesados

al dashboard. RNF-03 y RNF-04 están cubiertos por JWT y Circuit Breaker respectivamente, mientras que RNF-02 y RNF-05 son consecuencia directa de la estructura desacoplada y el uso del Repository Pattern en cada servicio.

En conclusión, el diseño propuesto cubre la totalidad de los requerimientos funcionales definidos y establece una base tecnológica sólida, escalable y mantenible para el crecimiento futuro de Grupo Cordillera.